GitHub Copilot for Cybersecurity Specialists

- Lesson 4: Security Code Review, Threat Modeling, and Auditing

Speaker Notes:

OPENING: Welcome back. In Lessons 1-3, we focused on building security - detecting vulnerabilities, implementing protocols, and automating tests. Lesson 4 shifts our focus to analyzing and verifying security. We're moving from creation to validation. This lesson teaches you how to use Copilot Chat and GitHub Advanced Security for comprehensive security analysis at scale. You'll learn interactive code review, automated threat modeling, compliance auditing, and intelligent dependency management. Let's get started.

Learning objectives

- Use Copilot Chat for interactive security code reviews and STRIDE threat modeling
- Generate automated security checklists and risk assessment reports
- Build custom security linters for configuration vulnerability detection
- Automate dependency analysis with AI-powered vulnerability prioritization

Speaker Notes:

FRAMER: This lesson is about turning Copilot into your security analyst. We're teaching the AI to think like a penetration tester while you maintain control and judgment.

BULLETS:

• Use Copilot Chat for interactive security code reviews and STRIDE threat modeling - Traditional code reviews take hours per file and threat modeling takes days per system. Copilot Chat enables conversational security analysis where you ask questions and refine findings in real-time. STRIDE analysis that took a workshop can now happen in minutes. The AI identifies threat vectors, you validate and prioritize.

• Generate automated security checklists and risk assessment reports - GHAS provides the data, Copilot transforms it into actionable documentation. You'll build prompts that generate CWE-mapped vulnerability reports, compliance checklists aligned to frameworks like OWASP Top 10, and risk assessment matrices. These aren't generic templates - they're contextualized to your actual codebase.

• Build custom security linters for configuration vulnerability detection - Generic linters catch common issues. Custom linters catch YOUR organization's specific anti-patterns. You'll use Copilot to generate ESLint plugins and Semgrep rules that enforce your security policies. Think: detecting hardcoded Azure connection strings, finding Kubernetes privilege escalations, or catching JWT validation bypass patterns.

• Automate dependency analysis with AI-powered vulnerability prioritization - Not all CVEs deserve equal attention. You'll build workflows where Copilot analyzes Dependabot alerts, assesses actual exploitability in your context, and generates pull requests with intelligent upgrade paths. This solves alert fatigue by focusing effort where real risk exists.

PRO TIP: The superpower here is combining GHAS's security telemetry with Copilot's reasoning. GHAS finds vulnerabilities, Copilot explains business impact and prioritizes remediation. In production, this reduces MTTR (mean time to remediation) from weeks to hours because you're not manually triaging hundreds of alerts.

Use Copilot Chat for security code review

- Conversational analysis replaces manual line-by-line auditing
- AI identifies auth/authz vulnerabilities, injection risks, and logic flaws
- STRIDE threat modeling generates comprehensive attack surface analysis
- 10x review velocity without compromising quality

Speaker Notes:

FRAMER: Here's the thing about traditional security code reviews - they're painfully slow and inconsistent. A senior AppSec engineer can review maybe 500 lines per hour if they're focused. Copilot Chat changes the economics entirely by providing instant conversational security analysis.

BULLETS:

• Conversational analysis replaces manual line-by-line auditing - Instead of reading every line, you describe your architecture to Copilot Chat and ask targeted security questions. "Does this authentication flow have session fixation risks?" or "Show me race conditions in this checkout process." The AI reads the entire codebase context and responds with specific findings. Wide World Importers reduced their code review time from 8 hours per PR to 45 minutes using this approach.

• AI identifies auth/authz vulnerabilities, injection risks, and logic flaws - Copilot Chat is trained on massive vulnerability datasets including OWASP patterns, CWE examples, and real-world exploits. It recognizes authentication bypass patterns like JWT signature validation skips, authorization flaws like IDOR (Insecure Direct Object References), SQL/NoSQL/LDAP injection opportunities, and business logic vulnerabilities like race conditions in payment processing. The key is providing enough architectural context in your prompt.

• STRIDE threat modeling generates comprehensive attack surface analysis - STRIDE is Microsoft's threat modeling framework: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. You give Copilot Chat your system architecture diagram or description, and it generates a complete STRIDE analysis with specific attack scenarios. Fabrikam generated a 15-page threat model in 20 minutes that would have taken their security team 2 weeks of workshops.

• 10x review velocity without compromising quality - This isn't hyperbole. The GitHub 2024 Security Report shows teams using AI-assisted review analyze 10x more code than manual review teams. Quality doesn't drop because humans still validate findings - you're just accelerating the discovery phase and eliminating false negatives from reviewer fatigue.

PRO TIP: When using Copilot Chat for security reviews, always start with an architecture context prompt. Paste your component diagram or describe your data flow before asking security questions. The more context you provide about trust boundaries, data classification, and deployment

architecture, the more accurate the threat analysis. I learned this from doing AppSec consulting - vague questions get vague answers. Specific architectural context gets specific, actionable findings.

Demo: Interactive security code review with Copilot Chat

- Analyze Express.js API for authentication and authorization vulnerabilities
- Use conversational prompts to identify JWT validation issues
- Generate STRIDE threat model for the API architecture
- Export findings as security review report

Speaker Notes:

FRAMER: Let's see Copilot Chat in action for security code review. I'll demonstrate analyzing a Node.js Express API for auth vulnerabilities and generating a threat model. Watch how the conversational approach finds issues traditional linters miss.

BULLETS:

• Analyze Express.js API for authentication and authorization vulnerabilities - We'll start with a realistic Express API that has JWT authentication and role-based access control. The code looks fine on surface inspection - it uses popular libraries like jsonwebtoken and express-validator. But there are subtle security flaws that Copilot Chat will identify through conversational analysis.

• Use conversational prompts to identify JWT validation issues - I'll show you how to structure security prompts: "Analyze the JWT validation in auth.middleware.js for signature verification bypass risks." Then follow-up: "Check if the token expiration is being enforced." This conversational approach mimics how a security consultant would audit code - asking probing questions and drilling deeper based on findings.

• Generate STRIDE threat model for the API architecture - Next, I'll paste a simple architecture description into Copilot Chat and request a STRIDE analysis. The AI will identify specific threats like: Spoofing through JWT token theft, Tampering with API payloads, Information Disclosure through error messages leaking system info, and Elevation of Privilege through RBAC bypass. You'll see how comprehensive this analysis is.

• Export findings as security review report - Finally, I'll show you how to structure a prompt that formats all findings into a security review report with CWE references, severity ratings, and remediation guidance. This becomes documentation for your security team and tracks remediation progress.

PRO TIP: During demos, I always save successful prompts to a "security-prompts" repo. Build a prompt library for common security review scenarios - auth analysis, injection scanning, cryptography review, etc. This turns ad-hoc analysis into a repeatable workflow. Contoso maintains 40+ security prompts they run on every major PR, catching issues before they reach production.

Generate automated security review checklists

- Transform GHAS vulnerability data into actionable security checklists
- Map findings to compliance frameworks (OWASP Top 10, CWE, NIST)
- Generate risk assessment matrices with business impact analysis
- Customize reports for different stakeholders (devs, security, executives)

Speaker Notes:

FRAMER: Raw vulnerability data is useless without context and prioritization. This is where Copilot transforms GHAS telemetry into reports that drive action across your organization.

BULLETS:

• Transform GHAS vulnerability data into actionable security checklists - GitHub Advanced Security surfaces hundreds or thousands of potential vulnerabilities through CodeQL, secret scanning, and Dependabot. You can't manually triage this volume. Copilot ingests the GHAS data and generates prioritized checklists organized by severity, exploitability, and business impact. Northwind reduced their vulnerability backlog from 847 issues to 67 critical items requiring immediate action.

• Map findings to compliance frameworks (OWASP Top 10, CWE, NIST) - Security findings need to map to compliance requirements for audit purposes. You'll build prompts where Copilot takes GHAS vulnerability data and maps each finding to specific OWASP categories, CWE identifiers, and NIST controls. This is critical for compliance reporting - auditors want to see evidence you're addressing framework requirements systematically.

• Generate risk assessment matrices with business impact analysis - Not all vulnerabilities have equal business impact. An SQL injection in your admin panel is catastrophic. A reflected XSS in your 404 page is low priority. Copilot can analyze your architecture context and generate risk matrices that consider likelihood and impact. The AI asks: "Is this endpoint internet-accessible? Does it process PII? Is it in the critical business flow?" This context drives intelligent prioritization.

• Customize reports for different stakeholders (devs, security, executives) - Developers need technical remediation steps. Security teams need CWE references and exploit details. Executives need business risk summaries and cost of remediation. One prompt, three different report formats. You'll see how to structure prompts that generate stakeholder-appropriate documentation from the same GHAS dataset.

PRO TIP: Build a GitHub Actions workflow that generates security reports on every commit to main. Use Copilot's API to analyze new GHAS findings and append them to a living security dashboard in your wiki. Adventure Works runs this automation - every sprint planning meeting starts with an auto-generated security report showing new vulnerabilities introduced in the last two weeks. This makes security visibility continuous instead of

quarterly.

Demo: Automated compliance reporting workflow

- Query GHAS API for CodeQL and Dependabot findings
- Use Copilot to generate OWASP Top 10 compliance report
- Create executive summary with risk metrics
- Automate weekly security digest via GitHub Actions

Speaker Notes:

FRAMER: Let's build a practical automation that security teams actually use in production. This demo shows end-to-end compliance reporting from GHAS data to executive dashboard.

BULLETS:

• Query GHAS API for CodeQL and Dependabot findings - We'll start by using GitHub's GraphQL API to fetch all active security alerts for a repository. I'll show you the query structure for pulling CodeQL SAST findings, secret scanning alerts, and Dependabot vulnerability reports. This data comes back as structured JSON that Copilot can process.

• Use Copilot to generate OWASP Top 10 compliance report - Here's where it gets interesting. I'll demonstrate a Copilot prompt that takes the raw GHAS JSON and maps each finding to OWASP Top 10 categories. The AI identifies "SQL Injection" as A03:2021 Injection, "Hardcoded Secrets" as A07:2021 Identification and Authentication Failures, etc. The output is a formatted markdown report showing your compliance posture.

• Create executive summary with risk metrics - For executive reporting, you don't show technical details - you show business impact. I'll use Copilot to transform the detailed findings into a one-page executive summary with metrics like: "23 critical vulnerabilities affecting payment processing, estimated remediation cost $15K, regulatory compliance risk HIGH." This is what CISOs need to see.

• Automate weekly security digest via GitHub Actions - Finally, I'll show you a GitHub Actions workflow that runs this entire pipeline automatically. Every Monday morning, the security team gets an email with the week's vulnerability status, trends over time, and auto-generated remediation tickets for critical issues. The whole workflow is 60 lines of YAML and a 10-line Copilot prompt.

PRO TIP: When building automated reports, always include trend data alongside current status. "We have 52 vulnerabilities" is less actionable than "We have 52 vulnerabilities, down from 78 last week, with 15 new findings introduced in PR #847." Context drives action. Tailwind Traders includes trend charts in every automated report - this helped them reduce their vulnerability count by 67% over 6 months because teams could see the impact of remediation work.

Build custom security linters for your environment

- Generic linters miss organization-specific anti-patterns
- Use Copilot to generate ESLint plugins for JavaScript security rules
- Create Semgrep rules detecting infrastructure misconfigurations
- Enforce security policies in pre-commit hooks and CI pipelines

Speaker Notes:

FRAMER: Off-the-shelf security tools catch common vulnerabilities. Custom linters catch YOUR organization's specific security mistakes. This is where Copilot becomes a force multiplier for security teams.

BULLETS:

• Generic linters miss organization-specific anti-patterns - ESLint and Semgrep ship with hundreds of security rules detecting SQL injection, XSS, and crypto mistakes. But they don't know YOUR organization decided JWT tokens must expire in 15 minutes max, or that all Azure Storage connections must use managed identity instead of connection strings, or that Kubernetes pods must never run as root in your production clusters. These are custom security policies that require custom linters.

• Use Copilot to generate ESLint plugins for JavaScript security rules - I'll demonstrate using Copilot to build an ESLint plugin that enforces custom security policies. For example, Contoso required that all API endpoints processing PII must have rate limiting enabled. Copilot generated a custom ESLint rule that scans Express.js route definitions and fails the build if rate limiting middleware is missing on PII endpoints. This became enforceable security policy.

• Create Semgrep rules detecting infrastructure misconfigurations - Semgrep is particularly powerful for infrastructure-as-code security. You'll use Copilot to generate Semgrep patterns that detect Terraform misconfigurations, Kubernetes privilege escalations, or AWS S3 buckets without encryption. Fabrikam uses custom Semgrep rules to catch 40+ organization-specific IaC security issues before they reach production.

• Enforce security policies in pre-commit hooks and CI pipelines - Custom linters are worthless if they're optional. We'll integrate these rules into pre-commit hooks using Husky and GitHub Actions CI pipelines. Developers get instant feedback on policy violations before they push code. The CI build fails if security policies aren't met. This shifts security left - catching issues at code-writing time instead of production.

PRO TIP: Start with your top 5 recurring security mistakes and build custom linters for those first. Don't try to create 100 rules at once - that's overwhelming. At Adventure Works, we identified their most common issues from security reviews: hardcoded secrets, missing RBAC checks, and unvalidated redirects. We built three custom linters for those patterns. After 6 months, those vulnerability categories dropped to near-zero because

developers got immediate feedback. Build linters for your actual problem patterns, not theoretical ones.

Demo: Custom security linter for Azure configuration

- Identify recurring Azure misconfiguration pattern (storage connection strings)
- Use Copilot to generate Semgrep rule detecting the anti-pattern
- Test the rule against vulnerable and secure code samples
- Integrate into GitHub Actions for automated enforcement

Speaker Notes:

FRAMER: Let's build a real custom security linter that solves an actual problem I see constantly in Azure consulting - developers hardcoding storage connection strings instead of using managed identity.

BULLETS:

• Identify recurring Azure misconfiguration pattern (storage connection strings) - The vulnerability: Developers paste Azure Storage connection strings directly into application code or config files. The secure pattern: Use DefaultAzureCredential with managed identity for authentication. This hardcoded credential pattern is a CWE-798 violation and a constant source of secret leaks. We're going to build a Semgrep rule that catches it.

• Use Copilot to generate Semgrep rule detecting the anti-pattern - I'll show you the exact prompt: "Generate a Semgrep rule that detects Azure Storage connection strings in JavaScript and C# code. The pattern should match AccountName= and AccountKey= in string literals." Copilot generates the YAML-based Semgrep rule with pattern matching that catches the vulnerability. We'll review the rule logic together.

• Test the rule against vulnerable and secure code samples - Security rules must have zero false positives or developers will disable them. I'll demonstrate testing the Semgrep rule against both vulnerable samples (hardcoded connection strings) and secure implementations (DefaultAzureCredential). The rule should catch the vulnerable code and pass the secure code. This validation is critical.

• Integrate into GitHub Actions for automated enforcement - Finally, we'll add the custom Semgrep rule to a GitHub Actions workflow that runs on every pull request. When a developer commits code with a hardcoded connection string, the build fails with a clear error message explaining the security policy and linking to documentation on proper managed identity usage. This makes security policy enforceable.

PRO TIP: When building custom linters, always include educational links in the error messages. Don't just say "Security violation detected" - say "Connection strings detected. Use managed identity instead. See: https://docs.microsoft.com/azure-identity." Wide World Importers found that adding documentation links to their custom linter errors reduced repeat violations by 80% because developers learned the secure pattern from the error message itself.

Automate dependency vulnerability management

- Dependabot alerts overwhelm teams with low-priority noise
- Use Copilot to analyze CVE exploitability in your specific context
- Generate intelligent upgrade paths considering breaking changes
- Automate pull request creation for high-priority vulnerabilities

Speaker Notes:

FRAMER: Real talk: Dependabot alert fatigue is real. You wake up Monday morning to 47 new vulnerability alerts. Which ones actually matter? Which are exploitable in your architecture? This is where Copilot's reasoning capabilities become critical.

BULLETS:

• Dependabot alerts overwhelm teams with low-priority noise - GitHub Dependabot is incredibly sensitive - it flags every published CVE affecting your dependency tree. Most aren't exploitable in your specific context. Example: Northwind got 200+ alerts for a lodash prototype pollution CVE. But they only used lodash for server-side data transformation with trusted inputs - the vulnerability wasn't reachable. Manually triaging 200 alerts took their security team 3 days. This doesn't scale.

• Use Copilot to analyze CVE exploitability in your specific context - Here's the breakthrough: Feed Copilot both the CVE details AND your architecture/usage context. "We use package X for Y functionality. CVE-2024-1234 affects feature Z. Is this vulnerability exploitable given our usage pattern?" Copilot analyzes the attack vector and your implementation, then provides an exploitability assessment. Fabrikam reduced their critical alert queue from 200 to 37 actually-exploitable vulnerabilities using this analysis.

• Generate intelligent upgrade paths considering breaking changes - Updating dependencies isn't just `npm update`. Major version bumps introduce breaking changes. Copilot can analyze your codebase usage of a vulnerable dependency, review the package's CHANGELOG, and generate an upgrade plan that identifies breaking changes and suggests code modifications. Adventure Works used this to upgrade a critical dependency with 15 breaking changes across their codebase in 4 hours instead of 2 weeks.

• Automate pull request creation for high-priority vulnerabilities - The ultimate workflow: Dependabot alerts → Copilot analyzes exploitability → High-priority vulnerabilities trigger auto-generated PRs with intelligent upgrade paths and code fixes. Medium-priority vulnerabilities get triaged to backlog. Low-priority alerts are documented but not actioned. This automation solves alert fatigue by focusing human effort where real risk exists.

PRO TIP: Build an exploitability decision tree that Copilot uses for consistency. Include questions like: "Is the vulnerable code path reachable from untrusted input? Does the application run with elevated privileges? Is the vulnerability network-exploitable?" This structured analysis ensures

consistent risk assessment. Tailwind Traders documented their exploitability criteria and feeds it to Copilot with every CVE analysis - this gave them consistent, defendable security decisions that satisfy audit requirements.

Demo: AI-powered dependency vulnerability workflow

- Fetch Dependabot alerts via GitHub API
- Use Copilot to assess CVE exploitability for critical alerts
- Generate pull request with dependency upgrade and code fixes
- Validate fixes pass security tests before merging

Speaker Notes:

FRAMER: This is my favorite demo in the course because it solves a problem every security team faces - dependency vulnerability overload. Let's build an end-to-end workflow that actually works in production.

BULLETS:

• Fetch Dependabot alerts via GitHub API - We'll start by querying GitHub's API for all active Dependabot alerts on a repository. The API returns vulnerability details including CVE identifier, affected package, vulnerable versions, and severity rating. I'll show you the GraphQL query that pulls this data efficiently.

• Use Copilot to assess CVE exploitability for critical alerts - Here's where intelligence enters the workflow. For each critical/high severity alert, I'll demonstrate feeding Copilot three inputs: (1) The CVE details from NVD, (2) Your codebase's usage of the vulnerable dependency, (3) Your architecture and trust boundaries. Copilot provides an exploitability assessment: "This RCE vulnerability is NOT exploitable in your context because the affected function is only called with admin-validated inputs behind authentication."

• Generate pull request with dependency upgrade and code fixes - For truly exploitable vulnerabilities, we'll use Copilot to generate an upgrade pull request. The AI analyzes the dependency's changelog, identifies breaking changes, updates package.json, and modifies consuming code to handle API changes. The PR includes test updates and a detailed description of what changed and why.

• Validate fixes pass security tests before merging - Automation without validation is dangerous. Every generated PR must pass your security test suite. I'll demonstrate running CodeQL, dependency scans, and custom security tests in GitHub Actions before allowing merge. The workflow only succeeds if security validation passes - no shortcuts.

PRO TIP: Implement a "Security PR Auto-Merge" policy for dependency updates that meet strict criteria: (1) Only patch/minor version bumps, (2) All tests pass, (3) No breaking changes detected, (4) CVE severity is critical/high. Contoso auto-merges about 40% of dependency PRs using this criteria, reducing manual review burden while maintaining security rigor. The other 60% requiring manual review get human attention because they're complex upgrades with breaking changes.

Key takeaways and next steps

- Copilot Chat enables 10x faster security code reviews
- Automated compliance reporting reduces audit preparation time
- Custom linters enforce organization-specific security policies
- AI-powered dependency analysis solves alert fatigue

Speaker Notes:

FRAMER: Let's recap what you learned in Lesson 4. These capabilities transform security from reactive to proactive - you're catching vulnerabilities before they reach production instead of firefighting after incidents.

BULLETS:

• Copilot Chat enables 10x faster security code reviews - You learned conversational security analysis that replaces slow manual line-by-line auditing. STRIDE threat modeling now takes minutes instead of days. Wide World Importers reduced code review time from 8 hours per PR to 45 minutes using this approach. The key: Providing architectural context to get accurate, specific security findings.

• Automated compliance reporting reduces audit preparation time - You can now transform raw GHAS vulnerability data into compliance reports mapped to OWASP, CWE, and NIST frameworks. Stakeholder-specific reports (technical, executive, audit) generate automatically. Northwind cut their quarterly compliance report preparation from 2 weeks to 4 hours using automated workflows.

• Custom linters enforce organization-specific security policies - Generic tools catch common issues. Custom linters catch YOUR specific anti-patterns. You learned to build ESLint plugins and Semgrep rules that enforce custom security policies, integrated into pre-commit hooks and CI for automated enforcement. This shifts security left - catching issues at code-writing time.

• AI-powered dependency analysis solves alert fatigue - You now have workflows that analyze CVE exploitability in your specific context, generate intelligent upgrade paths, and create automated PRs for high-priority vulnerabilities. Fabrikam reduced their critical alert queue from 200 to 37 actually-exploitable issues using exploitability analysis.

PRO TIP: The patterns you learned today are reusable across projects. Build a "security-automation" repository with your custom linters, report generation scripts, and exploitability decision trees. Every new project starts with this baseline security automation. This is how you scale security practices across an organization - through shared, proven automation patterns. In Lesson 5, we'll extend these concepts to compliance frameworks, incident response playbooks, and configuration management at scale. All course materials and code samples are available at timw.info/copilot-security.